

Informe del proyecto: Sistema de análisis de fenómenos de adolescencia basado en Seishun Buta Yarou

1. Introducción

El presente proyecto consiste en el análisis, diseño e implementación de un sistema orientado a objetos basado en el universo de Seishun Buta Yarou.

El sistema fue desarrollado con el objetivo de aplicar conceptos de Ingeniería de Software, Programación Orientada a Objetos, UML y principios SOLID, utilizando como contexto los fenómenos de adolescencia presentes en la historia.

A lo largo del proyecto se trabajó en diferentes etapas. Primero se identificaron las entidades y responsabilidades principales del sistema. Luego se elaboró el modelo de clases, se analizaron los principios SOLID, se definieron los casos de uso y se representó el comportamiento del sistema mediante un diagrama de secuencia.

Finalmente, todo el diseño fue llevado a una implementación en Java y se desarrolló una interfaz gráfica utilizando Java Swing. Esta interfaz permite ejecutar la simulación y visualizar los mensajes del sistema dentro de una ventana, utilizando además una imagen de Mai Sakurajima como fondo.

2. Objetivo general

Diseñar e implementar un sistema orientado a objetos basado en Seishun Buta Yarou, aplicando UML, principios SOLID y Java para representar personajes, fenómenos de adolescencia, investigaciones y relaciones entre personajes.

3. Objetivos específicos

Los principales objetivos del proyecto fueron:

- *Identificar las entidades principales del sistema.*
 - *Determinar las responsabilidades de cada entidad.*
 - *Diseñar las clases necesarias para representar el problema.*
 - *Definir atributos y tipos de datos para cada clase.*
 - *Establecer las relaciones entre las diferentes clases.*
 - *Aplicar los principios SOLID.*
 - *Elaborar un modelo de casos de uso.*
 - *Representar mediante un diagrama de secuencia el funcionamiento de la aplicación.*
 - *Implementar el diseño utilizando Java.*
 - *Incorporar abstracción, interfaces, herencia y polimorfismo.*
 - *Crear una interfaz gráfica utilizando Java Swing.*
 - *Mostrar los resultados de la simulación directamente en la interfaz.*
 - *Incorporar una imagen de fondo relacionada con el personaje de Mai Sakurajima.*
 - *Solucionar problemas relacionados con la transparencia de la interfaz.*
 - *Solucionar problemas de codificación de caracteres especiales.*
-

4. Descripción general del sistema

El sistema representa una aplicación destinada a simular la investigación de los denominados fenómenos de adolescencia.

El personaje principal utilizado en el sistema es Sakuta Azusagawa, quien cumple el papel de investigador. También se incorpora a Mai Sakurajima como uno de los personajes principales.

Los fenómenos se representan mediante una abstracción general denominada Fenómeno de Adolescencia. A partir de esta abstracción se pueden representar distintos tipos de fenómenos, como la invisibilidad social y los bucles temporales.

El sistema también incorpora una estructura destinada a representar investigadores y otra para representar las relaciones entre personajes.

La aplicación permite realizar una simulación en la cual se registran los personajes, se detecta un fenómeno, se realiza una investigación, se obtiene un diagnóstico, se resuelve el fenómeno, se analiza un segundo fenómeno relacionado con un bucle temporal y finalmente se analiza una relación entre personajes.

5. Análisis del problema

Para realizar el diseño se analizaron los elementos fundamentales del dominio.

En primer lugar se identificaron los personajes. Cada personaje posee información propia, como su identificación, nombre, edad, curso y escuela. En el caso de Mai también se incorpora su profesión.

Luego se identificaron los fenómenos de adolescencia. Debido a que existen diferentes tipos de fenómenos, se decidió utilizar una abstracción general que permita representar cualquier fenómeno sin que el resto del sistema tenga que conocer sus detalles internos.

También se identificó la necesidad de contar con un investigador. Para evitar que el sistema dependa directamente de una única implementación, se definió una interfaz para los investigadores y posteriormente una implementación concreta correspondiente a Sakuta.

Finalmente se identificó la necesidad de representar las relaciones entre personajes. Para esto se creó una entidad específica que permite definir un tipo de relación y una intensidad.

6. Modelo de clases

El modelo de clases está compuesto principalmente por personajes, fenómenos, investigadores y relaciones.

Las principales clases y componentes del sistema son:

- *Sakuta Azusagawa.*
- *Mai Sakurajima.*
- *Fenómeno de Adolescencia.*
- *Invisibilidad Social.*
- *Bucle Temporal.*
- *Investigador de Fenómenos.*
- *Investigador Sakuta.*
- *Relación.*
- *Tipo de Relación.*

Cada elemento tiene una responsabilidad determinada dentro del sistema.

7. Clase Sakuta Azusagawa

Sakuta Azusagawa representa al protagonista del sistema.

La clase almacena información relacionada con el personaje, incluyendo su identificación, nombre, edad, curso y escuela.

Además de almacenar información, Sakuta posee responsabilidades relacionadas con la observación e investigación de fenómenos.

Dentro del sistema, Sakuta cumple un papel fundamental porque es quien observa los fenómenos y participa en su investigación.

La decisión de separar esta responsabilidad en una clase específica permite mantener organizado el modelo y evita colocar la información de todos los personajes dentro de una única clase.

8. Clase Mai Sakurajima

Mai Sakurajima representa a uno de los personajes principales.

La clase contiene información como identificación, nombre, edad, curso, escuela y profesión.

En el contexto de la aplicación, Mai permite representar a uno de los personajes afectados por los fenómenos de adolescencia.

Su información se encuentra separada de la información de Sakuta, respetando el principio de responsabilidad única.

9. Fenómeno de Adolescencia

Fenómeno de Adolescencia representa el concepto general de los fenómenos que pueden aparecer en el sistema.

Se trata de una abstracción que permite trabajar con diferentes tipos de fenómenos.

La utilización de esta abstracción es importante porque permite que el sistema pueda manejar distintos fenómenos sin depender directamente de una clase concreta.

Entre las operaciones relacionadas con los fenómenos se encuentran el inicio del fenómeno, su resolución y la consulta de su estado.

De esta manera, el sistema puede trabajar con diferentes fenómenos utilizando una misma estructura general.

10. Invisibilidad Social

La Invisibilidad Social representa uno de los fenómenos utilizados en la simulación.

Este fenómeno posee una intensidad que permite representar el nivel o grado del fenómeno.

Durante la ejecución, se crea una instancia de Invisibilidad Social y esta es tratada por el sistema como un fenómeno de adolescencia.

Esto permite demostrar el uso de abstracción y polimorfismo, ya que el sistema trabaja con el concepto general de fenómeno en lugar de depender directamente de la clase específica.

11. Bucle Temporal

El Bucle Temporal representa otro tipo de fenómeno.

A diferencia de la invisibilidad social, este fenómeno está relacionado con un período de tiempo determinado.

Para representarlo se utilizan una fecha inicial y una fecha final.

El sistema permite iniciar el bucle, consultar si se encuentra activo y posteriormente resolverlo.

Este ejemplo demuestra que pueden existir diferentes comportamientos para distintos tipos de fenómenos, manteniendo una estructura común.

12. Investigador de Fenómenos

El Investigador de Fenómenos representa una interfaz dentro del sistema.

Su objetivo es definir el comportamiento que debe tener cualquier objeto que cumpla el papel de investigador.

Entre sus responsabilidades principales se encuentran investigar un fenómeno y obtener un diagnóstico.

La utilización de una interfaz permite separar el contrato de la implementación.

Esto significa que el sistema puede tener diferentes tipos de investigadores sin necesidad de modificar el resto del modelo.

13. Investigador Sakuta

Investigador Sakuta representa la implementación concreta del investigador.

Esta clase implementa las operaciones definidas por el Investigador de Fenómenos.

Su función principal consiste en analizar un fenómeno y generar un diagnóstico.

La separación entre la interfaz y la implementación permite aplicar principios de diseño orientado a objetos y facilita futuras ampliaciones del sistema.

Por ejemplo, en el futuro podrían existir diferentes investigadores con diferentes métodos de análisis sin necesidad de modificar la estructura general del programa.

14. Clase Relación

La clase Relación representa un vínculo entre personajes.

Esta clase contiene dos elementos principales: el tipo de relación y su intensidad.

El tipo permite determinar la naturaleza de la relación, mientras que la intensidad representa el nivel de dicha relación.

La clase también posee una operación destinada a generar una descripción de la relación.

Esto permite que la lógica relacionada con las relaciones se mantenga separada de las clases de los personajes.

15. Tipo de Relación

Tipo de Relación se representa mediante una enumeración.

La utilización de una enumeración permite establecer un conjunto controlado de valores posibles.

Entre los tipos considerados se encuentra la relación romántica, aunque el modelo puede ampliarse para incluir otros tipos, como amistad o relación familiar.

Esta solución evita depender de cadenas de texto arbitrarias y hace que el sistema sea más consistente.

16. Aplicación de los principios SOLID

Uno de los objetivos principales del proyecto fue aplicar los principios SOLID.

Estos principios permiten construir sistemas más organizados, mantenibles, extensibles y fáciles de comprender.

16.1 Principio de Responsabilidad Única

El principio de responsabilidad única establece que una clase debe tener una única responsabilidad principal.

Este principio se aplica separando las diferentes funciones del sistema.

Por ejemplo, Sakuta se encarga de representar y realizar acciones relacionadas con Sakuta. Mai representa la información de Mai. Los fenómenos se encargan de su propio comportamiento y la clase Relación se encarga de representar las relaciones.

De esta manera se evita crear una única clase que contenga toda la lógica del sistema.

16.2 Principio Abierto/Cerrado

El principio abierto/cerrado indica que una clase debe estar abierta a la extensión pero cerrada a modificaciones innecesarias.

Este principio se aplica principalmente mediante la abstracción de los fenómenos.

Gracias a esto, el sistema puede incorporar nuevos tipos de fenómenos sin tener que modificar toda la estructura existente.

Por ejemplo, sería posible incorporar en el futuro nuevos fenómenos relacionados con memoria, sueños, identidad o percepción.

16.3 Principio de Sustitución de Liskov

Este principio establece que una implementación derivada debe poder utilizarse donde se espera la abstracción correspondiente.

En el proyecto esto se observa en los diferentes tipos de fenómenos.

Tanto la Invisibilidad Social como el Bucle Temporal pueden ser tratados como Fenómenos de Adolescencia.

Esto permite que el resto del sistema pueda trabajar con ambos utilizando la misma abstracción.

16.4 Principio de Segregación de Interfaces

Este principio busca evitar interfaces demasiado grandes que obliguen a las clases a implementar funcionalidades innecesarias.

En el proyecto se utiliza una interfaz específica para los investigadores.

La interfaz contiene únicamente las operaciones relacionadas con la investigación de fenómenos.

Esto mantiene el diseño sencillo y evita responsabilidades innecesarias.

16.5 Principio de Inversión de Dependencias

Este principio indica que las partes de alto nivel deberían depender de abstracciones y no de implementaciones concretas.

La utilización de la interfaz de Investigador de Fenómenos permite que el sistema trabaje con el concepto general de investigador.

La implementación concreta corresponde a Investigador Sakuta, pero podría reemplazarse posteriormente por otro investigador.

Esto reduce el acoplamiento entre los componentes.

17. Casos de uso

El análisis de casos de uso permitió identificar las principales acciones que puede realizar el usuario dentro del sistema.

El actor principal es el usuario de la aplicación.

Entre las principales funcionalidades se encuentran:

- *Ejecutar la simulación.*

- *Registrar o crear personajes.*
 - *Detectar un fenómeno.*
 - *Investigar un fenómeno.*
 - *Obtener un diagnóstico.*
 - *Resolver un fenómeno.*
 - *Iniciar un bucle temporal.*
 - *Consultar el estado de un fenómeno.*
 - *Analizar una relación.*
 - *Visualizar los resultados.*
 - *Limpiar el registro de la interfaz.*
-

18. Caso de uso: ejecutar simulación

El usuario inicia la simulación desde la interfaz gráfica.

El sistema limpia el registro anterior y comienza a crear los objetos necesarios.

Primero se crean Sakuta y Mai.

Posteriormente se crea un fenómeno de Invisibilidad Social.

Sakuta observa el fenómeno y participa en su investigación.

Luego interviene el investigador y se obtiene un diagnóstico.

Después se intenta resolver el fenómeno.

Una vez terminado este proceso, se crea un Bucle Temporal.

El sistema inicia el bucle, consulta su estado y posteriormente lo resuelve.

Finalmente se crea una relación de tipo romántico y se muestra su descripción.

Al finalizar todas las operaciones, el sistema informa que la simulación ha terminado.

19. Diagrama de secuencia y comportamiento

El diagrama de secuencia permitió estudiar el orden temporal en que interactúan los diferentes objetos.

El flujo comienza con el usuario, que solicita ejecutar la simulación.

La interfaz principal recibe la solicitud y comienza la ejecución.

A continuación se crean los personajes.

Luego se crea el fenómeno.

Sakuta observa e investiga el fenómeno.

El investigador realiza su análisis y devuelve un diagnóstico.

El sistema muestra el diagnóstico.

Posteriormente el fenómeno es resuelto.

Después se crea un nuevo fenómeno, correspondiente al Bucle Temporal.

El sistema lo inicia, consulta su estado y finalmente lo resuelve.

Por último, se crea una relación y se obtiene su descripción.

Todos los resultados son enviados a la interfaz gráfica para que el usuario pueda visualizarlos.

20. Implementación en Java

Una vez finalizado el diseño UML, se realizó la implementación utilizando Java.

La implementación utiliza Programación Orientada a Objetos para representar las diferentes entidades del sistema.

Se utilizaron clases, clases abstractas, interfaces, enumeraciones, herencia y polimorfismo.

El programa tiene una clase principal denominada Main, que controla la interfaz gráfica y la ejecución de la simulación.

Sin embargo, la lógica principal relacionada con los personajes, fenómenos, investigadores y relaciones se mantiene separada en sus respectivas clases.

Esto permite mantener una organización coherente entre el diseño UML y la implementación.

21. Interfaz gráfica

Posteriormente se decidió mejorar el sistema incorporando una interfaz gráfica.

Para esto se utilizó Java Swing.

La interfaz se diseñó como una ventana principal que contiene un título, un subtítulo, un registro de actividad, botones de control y una imagen de fondo.

El objetivo fue transformar la aplicación de una simple ejecución por consola en una aplicación visual más agradable y cercana a una aplicación real.

22. Registro del sistema

La interfaz posee un área destinada a mostrar los mensajes generados durante la ejecución.

En lugar de mostrar los mensajes únicamente en la consola tradicional de Java, se realizó una redirección para que los mensajes aparezcan dentro de la ventana.

Esto permite mantener los mensajes originales de las diferentes clases sin tener que modificar toda la lógica del programa.

El usuario puede visualizar en tiempo real las diferentes etapas de la simulación.

23. Botón de ejecución

La interfaz posee un botón destinado a iniciar la simulación.

Cuando el usuario lo presiona, se limpia el registro anterior y comienza una nueva ejecución.

La simulación se ejecuta de manera independiente del hilo principal de la interfaz, evitando que la ventana se bloquee durante el procesamiento.

24. Botón de limpieza

También se incorporó un botón para limpiar el registro.

Su función es eliminar los mensajes mostrados actualmente en pantalla y dejar el área preparada para una nueva ejecución.

Esto facilita las pruebas y permite ejecutar varias simulaciones sin tener que cerrar y abrir nuevamente el programa.

25. Imagen de fondo

Como parte del diseño visual se incorporó una imagen de Mai Sakurajima como fondo.

La aplicación intenta cargar la imagen desde los recursos del proyecto y, si no está disponible allí, intenta localizarla externamente.

El objetivo es que la imagen pueda ser fácilmente reemplazada por otra sin necesidad de modificar la lógica principal del sistema.

26. Transparencia de la interfaz

Durante las pruebas se detectó que el panel utilizado para mostrar la consola ocultaba completamente la imagen de fondo.

Para solucionar este problema se modificó la interfaz para utilizar paneles transparentes y fondos semitransparentes.

De esta manera, la imagen permanece visible detrás de la información.

La consola utiliza una capa oscura con cierto nivel de transparencia para mejorar la lectura del texto sin ocultar completamente la imagen.

Este cambio permite combinar funcionalidad y estética.

27. Problema con los caracteres especiales

Durante la ejecución se detectó otro problema relacionado con los caracteres especiales del idioma español.

Los caracteres con tilde y la letra ñ podían mostrarse incorrectamente.

Esto sucedía debido a la forma en que se estaban procesando los bytes enviados mediante la salida estándar.

El problema era especialmente visible en palabras como fenómeno, investigación, resolución, diagnóstico y análisis.

28. Solución de codificación

Para solucionar este inconveniente se modificó el mecanismo de redirección de la salida estándar.

En lugar de interpretar cada byte individualmente como un carácter, se decidió acumular los bytes y posteriormente interpretarlos utilizando la codificación UTF-8.

Esto permite que los caracteres especiales sean interpretados correctamente antes de ser enviados al área de texto de la interfaz.

De esta forma, la aplicación puede mostrar correctamente textos en español.

29. Ejecución de la simulación

El flujo final de ejecución del sistema es el siguiente:

Primero se inicia la aplicación.

Luego se muestra la ventana principal con la imagen de fondo.

El usuario puede presionar el botón para ejecutar la simulación.

El sistema crea a Sakuta Azusagawa y Mai Sakurajima.

Después se registra la información de los personajes.

A continuación se detecta un fenómeno de Invisibilidad Social.

Sakuta observa el fenómeno.

Posteriormente se realiza la investigación.

El Investigador Sakuta analiza el fenómeno.

Se obtiene un diagnóstico.

El fenómeno es resuelto.

Luego se crea un Bucle Temporal.

El sistema inicia el bucle y comprueba si se encuentra activo.

Después el bucle es resuelto.

Finalmente se crea una relación de tipo romántico con una intensidad determinada.

El sistema obtiene la descripción de la relación.

Una vez finalizado todo el proceso, se informa que la simulación terminó.

30. Tecnologías utilizadas

Para el desarrollo se utilizaron principalmente las siguientes tecnologías y conceptos:

Java como lenguaje de programación principal.

Programación Orientada a Objetos como paradigma de desarrollo.

UML como herramienta para el análisis y diseño.

Java Swing para la creación de la interfaz gráfica.

AWT para determinados componentes gráficos y operaciones relacionadas con la imagen.

LocalDate para representar fechas utilizadas en el fenómeno de Bucle Temporal.

Enumeraciones para representar los tipos de relaciones.

Interfaces para definir contratos de comportamiento.

Clases abstractas para representar fenómenos generales.

Hilos para ejecutar la simulación sin bloquear la interfaz.

UTF-8 para manejar correctamente los caracteres especiales.

31. Ventajas de la arquitectura desarrollada

El sistema presenta una estructura modular.

Cada clase posee responsabilidades concretas y las diferentes partes del sistema pueden evolucionar de forma independiente.

La utilización de una abstracción para los fenómenos permite incorporar nuevos tipos sin modificar toda la aplicación.

La utilización de una interfaz para los investigadores permite agregar nuevas implementaciones.

La clase de interfaz gráfica se ocupa principalmente de la interacción visual, mientras que las clases del dominio representan los conceptos propios del sistema.

Además, el uso de SOLID facilita el mantenimiento y la ampliación del proyecto.

32. Posibles ampliaciones

El proyecto puede continuar desarrollándose incorporando nuevas funcionalidades.

Una posible mejora sería agregar una base de datos para almacenar los personajes, fenómenos, investigaciones y relaciones.

También podrían incorporarse nuevos tipos de fenómenos de adolescencia.

Otra posibilidad sería permitir que el usuario seleccione manualmente un personaje y un fenómeno desde la interfaz.

También podría incorporarse un sistema de historial para consultar investigaciones anteriores.

La interfaz podría incluir diferentes pantallas para personajes, fenómenos, relaciones e investigaciones.

También podrían agregarse animaciones, indicadores de estado y diferentes imágenes dependiendo del fenómeno seleccionado.

33. Conclusión

El proyecto permitió aplicar de manera práctica diferentes conceptos fundamentales de Ingeniería de Software y Programación Orientada a Objetos.

A partir del universo de Seishun Buta Yarou se realizó un análisis de las entidades principales y se diseñó una estructura orientada a objetos que permite representar personajes, fenómenos de adolescencia, investigadores y relaciones.

El diseño fue desarrollado siguiendo los principios SOLID, buscando principalmente reducir el acoplamiento, separar responsabilidades y facilitar futuras ampliaciones.

También se trabajó con UML mediante el desarrollo del modelo de clases, los casos de uso y el análisis de secuencia de las principales operaciones del sistema.

Posteriormente, el diseño fue implementado en Java utilizando abstracción, herencia, interfaces, polimorfismo y enumeraciones.

Como etapa final se desarrolló una interfaz gráfica utilizando Java Swing. Esta interfaz permite ejecutar la simulación, visualizar los mensajes del sistema, limpiar el registro y mostrar una imagen de Mai Sakurajima como fondo.

Durante la implementación surgieron problemas relacionados con la transparencia de los componentes y con la codificación de caracteres especiales. Ambos problemas fueron solucionados mediante el uso de componentes gráficos transparentes, fondos semitransparentes y una correcta interpretación de los datos mediante UTF-8.

En conclusión, el proyecto permitió integrar análisis, diseño UML, principios SOLID, programación orientada a objetos e implementación gráfica en una única aplicación. Además de cumplir con los objetivos planteados, la estructura obtenida permite continuar ampliando el sistema en el futuro con nuevos personajes, fenómenos, investigadores y funcionalidades.